

Wady i zalety Kubernetes

Realistyczne spojrzenie z perspektywy QA

Kubernetes pomaga, ale nie jest magią.



Cel modułu

zrozumieć, dlaczego firmy używają Kubernetes
zobaczyć, jakie problemy Kubernetes rozwiązuje dobrze
nazwać koszty i trudności związane z platformą
wiedzieć, co jest istotne z perspektywy QA
odróżnić zalety technologii od realiów utrzymania



Co Kubernetes robi dobrze?

self-healing

skalowanie

rollout

izolacja

Kubernetes jest dobry tam, gdzie aplikacje mają działać stale,
powtarzalnie i w wielu środowiskach.



Zaleta: self-healing

```
kubectl delete pod <pod-name>  
kubectl get pods -w
```

Jeśli Pod znika, Deployment pilnuje oczekiwanej liczby replik i tworzy nowy.

Dla QA: warto wiedzieć, czy problem jest trwały, czy platforma już go odtworzyła.



Zalety: skalowanie i rollout

```
kubectl scale deployment qa-web --replicas=3  
kubectl rollout status deployment/qa-web  
kubectl rollout history deployment/qa-web
```

łatwe zwiększanie liczby replik

kontrolowane wdrażanie nowych wersji

możliwość obserwacji procesu rollout



Zaleta: deklaratywność

```
kubectl apply -f deployment.yaml  
kubectl apply -f service.yaml
```

Opisujemy oczekiwany stan w YAML, a Kubernetes próbuje doprowadzić środowisko do tego stanu.

Dla QA: łatwiej porównać środowiska, bo konfiguracja jest jawna.



Zaleta: izolacja środowisk

dev
namespace

qa
namespace

prod
namespace

Namespace pomaga oddzielić zasoby, konfigurację i dostęp do aplikacji.



Co Kubernetes komplikuje?

więcej warstw między użytkownikiem a aplikacją
więcej miejsc, w których może wystąpić błąd
trudniejsze debugowanie niż przy pojedynczym procesie
konieczność rozumienia sieci, DNS, storage i konfiguracji
łatwo pomylić problem aplikacji z problemem platformy



Wada: wiele warstw diagnostyki

aplikacja

kontener

Pod

Service

DNS

network

node

Dla QA najważniejsze jest zawęzić problem: aplikacja, konfiguracja, Service czy platforma.



Wada: koszt utrzymania

klaster trzeba aktualizować i monitorować
trzeba obsłużyć certyfikaty, DNS, sieć i storage
potrzebne są logi, metryki i alerty
problemy platformowe wymagają wiedzy operatorskiej
sam Kubernetes nie zastępuje dobrych praktyk aplikacyjnych



Kiedy Kubernetes ma sens?

Ma sens, gdy:

jest wiele usług
potrzebne są rollouty i skalowanie
środowiska mają być powtarzalne
zespół ma kompetencje platformowe

Może być overkill, gdy:

aplikacja jest bardzo prosta
brakuje zespołu utrzymaniowego
nie ma potrzeby skalowania
koszt przewyższa zysk



Perspektywa QA

QA nie musi budować klastra, ale powinien rozumieć podstawowe obiekty

najważniejsze są statusy, logi, eventy, Service i konfiguracja

warto umieć odtworzyć problem i zebrać konkretne dowody

dobry raport z błędu powinien zawierać namespace, Pod, logi i opis objawu

```
kubectl get pods && kubectl describe pod && kubectl logs
```



Do zapamiętania

Zaleta

automatyzacja, powtarzalność, self-healing

Wada

złożoność, koszty utrzymania, trudniejszy debugging

Kubernetes jest platformą do zarządzania aplikacjami.
Im więcej daje automatyzacji, tym więcej trzeba rozumieć podczas diagnozy.

